Pointer-First Agent Constitution for Stela: Architectural Analysis and Governance Report

1. Introduction: The Imperative of Statutory Governance in AI Development

The Stela system represents a fundamental departure from traditional, probabilistic software engineering methodologies, moving toward a deterministic, provenance-forward architecture designed specifically for the era of repo-resident Artificial Intelligence. As AI agents transition from passive coding assistants to autonomous "Contractors" capable of executing complex logic, the primary challenge shifts from code generation to entropy management. In a stateless system where the agent executes tasks without persistent memory, the repository itself must function as the entire cognitive architecture. This necessitates a transition from implicit tribal knowledge to "Statutory Governance," where the rules of engagement are not merely suggestions but physical laws enforced by the repository's structure.

The analysis of the Stela system, specifically through the lens of **DP-OPS-0039** and the **Pointer-First Constitution**, reveals a rigorous attempt to solve the "Drift" problem—the tendency of AI systems to hallucinate deviations from established patterns over time. By establishing a **Policy of Truth (PoT)** as the single source of authority, Stela inverts the typical developer-tool relationship. Here, the tool (the repo) governs the developer (the agent). This report provides an exhaustive examination of the Stela architecture, focusing on the staffing protocols, behavioral logic standards, and the operational friction observed during the transition to a pointer-first dashboard, specifically regarding the "Dirty State" paradox and the failure of fencing mechanics in AI outputs.[1]

1.1 The Philosophy of Context as File

The central axiom of the Stela architecture is that **Context is not memory; it is a file**.[1] In a stateless environment, an AI agent cannot "remember" the project's history, architectural decisions, or governance rules from one session to the next. It must be "rehydrated" with a precise, deterministic set of artifacts at the start of every session. If a rule, file, or constraint is not explicitly present in the loaded context, it effectively does not exist. This philosophy drives the **Filing Doctrine** defined in PoT.md, which segments the repository into strictly defined jurisdictions to prevent context pollution and hallucination.

The transition to a "Pointer-First" constitution acknowledges the limitations of the AI context window. Rather than dumping the entire repository into the agent's working memory, the system relies on a network of pointers. The TASK.md file serves as the active dashboard,

pointing to the PoT.md for law, the SoP.md for history, and the docs/MAP.md for terrain. This minimizes the cognitive load on the agent while maximizing adherence to the Single Source of Truth (SSOT). However, as evidenced by the thinking logs for **DP-OPS-0039** [1], this transition creates friction when automated tools (linters) and AI agents (Contractors) differ in their interpretation of these pointers, particularly regarding the formatting of Dispatch Packets (DPs) and the handling of uncommitted "dirty" states in the repository.

2. The Policy of Truth (PoT): Constitutional Analysis

The **Policy of Truth (PoT.md)** is the immutable core of the Stela system. It is not a guideline; it is a constitution that defines the physical laws of the repository.[1] The document explicitly states that it "does not request compliance; it prevents action that violates policy." This distinction is critical for AI alignment. An AI Contractor cannot choose to follow the rules; the rules are constraints on its operation, enforced by the Integrator and the CI/CD pipeline (repo-gates).

2.1 The Filing Doctrine

The Filing Doctrine, defined in Section 1.1 of PoT.md, establishes the physical geography of the repository. This segmentation is crucial for "Anti-Drift Governance" because it restricts where an agent can read and write, preventing the contamination of canonical documentation with operational debris.

Directory	Type	Purpose	Governance Implication
ops/	**Run**	Runtime Kernel, Enforcement	Contains binaries (ops/bin/), manifests, and automation. This is the enforcement arm of the constitution. Agents typically execute *from* here but rarely write *to* here without explicit authorization.
docs/	**Explain**	Legislation,	Contains the

		Manuals	Constitution (PoT.md), History (SoP.md), and Explanations (MANUAL.md). This is the Source of Truth. Writes here trigger high-level policing workflows (sop_policing.yml).
projects/	**Work**	Payload, Product Code	The only area where "product" code lives. All other areas are "platform." This isolation prevents payload code from importing platform logic, maintaining a strict separation of concerns.
storage/	**Trash**	Transient, Non-Canon	Buffer for local artifacts, dumps, and handoffs. It is explicitly non-canonical. Agents dump state here (storage/dumps/) but must never treat files here as permanent truth.[1]

2.2 Axioms of Governance

Section 1.2 of PoT.md outlines the axioms that govern all operations. These axioms are designed to counter specific failure modes of Large Language Models (LLMs).[1]

- **Precedence:** PoT.md is the final authority. If a conflict exists between the prompt and the file, the file wins. This protects against "Prompt Injection" or "Ghost Canon" where an agent follows a hallucinated rule.

- **SSOT (Single Source of Truth):** Only one canonical file is allowed per domain. Other mentions must be pointers. This prevents "Knowledge Forking," where conflicting versions of a rule exist in different documents.
- **Reuse-First:** Operators must search ops/ for existing templates before creating new artifacts. This counters the LLM's tendency toward "Tool Sprawl"—creating new scripts (e.g., test.sh, run_tests.sh) instead of using the canonical tools/verify.sh.
- **Context Hazard:** A specific "dirty" state is defined as a "Context Hazard," which occurs if library directories (docs/library/agents, docs/library/tasks, docs/library/skills) are included in the global context manifest (ops/lib/manifests/CONTEXT.md).[1] This rule enforces "Just-In-Time" (JIT) knowledge loading, preventing the agent from being overwhelmed by irrelevant instructions.

2.3 Staffing Protocol and Jurisdiction

The Staffing Protocol (Section 4.1) defines the hierarchy of authority, which is strictly enforced to prevent AI-induced corruption of the repository.[1]

- **Operator (Human):** The Sovereign. The Operator owns final decisions, approvals, and secrets. Crucially, the Operator performs all commits, pushes, and merges. This "Human-in-the-Loop" (HITL) commit authority is the ultimate fail-safe. It decouples the *generation* of code (AI) from the *ratification* of code (Human), creating a mandatory "Reality Check" at the end of every work cycle.
- **Integrator (Lead AI):** The Auditor. The Integrator maintains governance, structural integrity, and auditing. It generates Dispatch Packets (DPs) and detects "Drift." It effectively acts as the "Internal Affairs" division, policing the work of the Contractor.
- **Contractor (Guest AI):** The Executor. The Contractor executes specific logic tasks and drafts implementation details within a defined scope. It operates under strict behavioral constraints, forbidden from inferring intent beyond the explicit instructions in the DP.

3. Behavioral Logic Standard: Linguistic Precision as Governance

The Behavioral Logic Standard (Section 4.2 of PoT.md) imposes strict constraints on how AI agents communicate and report.[1] These are not stylistic choices; they are functional requirements for technical auditing.

3.1 Linguistic Precision

- **Contraction Prohibition:** The use of contractions (e.g., "don't," "can't," "won't") is strictly forbidden across all scopes. This ensures rhythmic precision. In natural language processing, contractions can introduce ambiguity or informal tone that masks imprecise thinking. By forcing formal language ("do not," "cannot"), Stela forces the agent to adopt a formal cognitive stance, reducing the likelihood of "vibes-based" outputs.[1]

- **Quantitative Reporting:** Agents must specify exact deviations (e.g., "execution path deviated by 14.7%") rather than subjective descriptions ("it didn't work well"). This is essential for the "Integrator" role, which requires precise data to detect drift. "Vibes" are subjective; numbers are auditable.
- **Absolute Literalism:** Agents must interpret instructions with total fidelity. If a command is ambiguous, they must "stop and ask" rather than guess. This counters the RLHF (Reinforcement Learning from Human Feedback) training of most LLMs, which biases them toward being "helpful" by inferring user intent. In Stela, inferring intent is a violation of protocol; the agent must execute only what is written.

3.2 Operational Directives and Stop Conditions

The system defines explicit "Stop Conditions" to prevent cascading failures.[1]

- **Anti-Drift Governance:** Logic or files not aligned with PoT.md coordinates represent a system failure. If detected, the agent must stop.
- **Logic Conflict Resolution:** If a task violates PoT.md invariants, the agent must cease operations until the Operator redefines the parameters. This prevents the agent from being coerced into violating the constitution by a poorly formed prompt.
- **Equilibrium Maintenance:** A task is considered complete *only* when the system state reaches equilibrium and SoP.md (the history ledger) is updated. This links code changes to documentation updates, ensuring the history ledger never drifts from the actual code state.

4. The Pointer-First Dashboard: TASK.md Refinement

The transition to a "Pointer-First" constitution is physically manifested in the refactoring of TASK.md. This file has evolved from a simple to-do list into a sophisticated, dynamic dashboard that serves as the "Contract" between the Operator and the Contractor.[1]

4.1 From Template to Dashboard

Previously, TASK.md contained static templates for tasks. In the new architecture (DP-OPS-0038/0039), it serves as a "Pointer-First Dashboard".[1]

- **Session State (The Anchor):** This section anchors the agent to the current reality. It points to the specific OPEN artifact (storage/handoff/OPEN-*.txt) containing the git branch and hash. This prevents the agent from hallucinating that it is working on a different branch or commit.[1]
- **Logic Pointers (The Law):** Instead of repeating rules, this section contains pointers to PoT.md and docs/GOVERNANCE.md. This reinforces the SSOT axiom—there is only one version of the law, and TASK.md merely points to it.
- **Current Dispatch Packet (DP):** This is the active contract. It defines the specific scope, execution plan, and allowable files for the current session.

4.2 The Decimal Schema Tension

The refinement of TASK.md revealed a tension between the human-readable dashboard format and the machine-readable requirements of the linting tools. The dp lint tool (tools/lint/dp.sh) expects a strict "Decimal Schema" for headings (e.g., ## 3.1 Freshness Gate, ## 3.2 Required Context Load).[1]

- **Refinement Logic:** The analysis in DP-OPS-0039-CHATGPT-THINKING.md shows the Integrator grappling with this constraint. While the dashboard format is "pointer-first," the internal structure of the DP *must* strictly follow the decimal schema to pass the automated gates.
- **Numbering Adjustments:** The analysis led to specific refinements, such as removing the trailing dot in "4. Closeout" (becoming "4 Closeout") and standardizing "## 5 Work Log" to ensure the regex patterns in the linter would correctly identify the sections.[1] This demonstrates that in Stela, **tooling dictates documentation**; the manual must conform to the linter, not the other way around.

5. The "Dirty State" Paradox and Operational Friction

One of the most significant operational frictions identified in the analysis is the "Dirty State" paradox involving TASK.md and the ops/bin/open binary. This conflict highlights the nuances of managing a "Living Surface" within a version-controlled repository.[1]

5.1 The Logic of ops/bin/open

The ops/bin/open binary is the session initiator. Its primary function is to "rehydrate" the context for the AI agent. To do this effectively, it must inject the *current* session state—the Active Branch, the Base HEAD hash, and the Intent—directly into the TASK.md file.[1]

- **Mechanism:** The script calculates the git state (git rev-parse --short HEAD) and overwrites the placeholders in TASK.md with these real-time values.
- **Result:** This action immediately modifies a tracked file (TASK.md), causing the git working tree to become "dirty." The git status --porcelain command reports M TASK.md.[1]

5.2 The Worker's Complaint

The AI Contractor is programmed (via PoT.md and copilot-instructions.md) to value "Equilibrium" and "Anti-Drift." It is trained to verify a "Clean State" before beginning work to ensure provenance.

- **The Conflict:** When the Contractor starts execution, it runs the "Freshness Gate" check. It sees that TASK.md is modified (M TASK.md). It interprets this as "Drift"—an uncommitted change that violates the clean state axiom. Consequently, the worker "complains" or halts, requesting that the state be cleaned before proceeding.[1]
- **The Paradox:** The dirty state is *intentional*. TASK.md is designed to be a mutable

dashboard that reflects the volatile state of the session. However, the PoT.md defines it as a "Canon Surface," implying it should be static and versioned. The listing of TASK.md in .gitignore as a "Local task template" further complicates this, creating a "Schrödinger's File" scenario where it is both canon (tracked) and local (ignored).[1]

5.3 Resolution Strategy

The friction arises from a clash between the **Static Truth** of the repository (git history) and the **Dynamic Truth** of the session (active intent). The recommended resolution, implied by the analysis of the gate surfaces, is to treat the M TASK.md state as a sanctioned exception during the active session window. The task-policing workflow [1] enforces structural updates, but the session-specific data injection by ops/bin/open must be recognized by the Contractor as a valid "Operational State" rather than "System Drift."

6. Forensic Analysis of DP-OPS-0039: Gemini vs. ChatGPT

The comparative analysis of the thinking logs for **DP-OPS-0039** reveals critical differences in how different AI models interpret and execute the Stela constitution, specifically regarding the "Fencing" of Dispatch Packets.[1]

6.1 Gemini Thinking: Structural Rigor

The Gemini model's approach to **DP-OPS-0039** [1] was characterized by rigorous structural mapping.

- **Mapping:** It meticulously mapped the legacy "Freshness Gate" sections to the new "Scope Boundary" and "Execution Plan" hierarchy.
- **Validation:** It identified a branch naming error (0038 vs 0039) and corrected it based on context logic.
- **Fencing:** Gemini adhered to the markdown fencing requirements, ensuring that the DP was contained within a verifiable block.

6.2 ChatGPT Thinking: The Fencing Failure

The ChatGPT model [1] demonstrated a "Fencing Failure." While it correctly analyzed the heading strings and the decimal schema, it failed to wrap the final Dispatch Packet in a markdown code block (triple backticks).

- **The Error:** The Operator observed: "CHATGPT DID NOT FENCE IN DP IN A CODE BOX."
- **The Cause:** The thinking logs suggest that ChatGPT became preoccupied with the *internal* syntax of the DP (dots in headers, separators) and lost track of the *external* container requirement. It focused on whether --- separators were allowed *inside* the DP, but forgot to put the DP *inside* a fence.

- **The Implication:** An unfenced DP blends with the conversational text of the chat. If an Operator copies the output, they might accidentally include non-canonical conversational filler ("Here is the DP you requested...") into the TASK.md file. This violates the "Absolute Literalism" and "SSOT" axioms. The DP must be a discrete, copy-pasteable artifact.

6.3 Refinement of the Closeout Routine

Both models addressed the need to harden the **Closeout** routine. The analysis revealed that tasks were failing silently because they executed their logic but never formally "closed out" by updating the SoP.md ledger.

- **The Fix: DP-OPS-0039** mandates that every B-TASK file in the library must include an explicit final step in its Execution Logic: "Execute Closeout per TASK.md Section 4".[1]
- **Quantitative Rigor:** The new standard requires "Quantitative Reporting." It is no longer sufficient to say "Verification passed." The agent must capture the specific output of the verification tool (e.g., 5 files linted, 0 errors) and paste it into the RESULTS artifact.

7. The Tooling Ecosystem: The Ops Kernel

The ops/ directory functions as the "Runtime Kernel" of the Stela system. It contains the binaries that enforce the constitution. These are not helper scripts; they are the codified laws of the repository.[1]

7.1 ops/bin/open (The Session Initiator)

- **Function:** Generates the "Open Prompt" and freshness gate.
- **Critical Role:** It is the mechanism that injects reality into the agent. By capturing the git status and HEAD hash, it prevents the agent from operating on stale or hallucinated code.
- **Gate Surface:** It is a "Gate Surface," meaning any change to this binary requires a corresponding update to TASK.md.[1]

7.2 ops/bin/dump (The Observer)

- **Function:** Captures the platform state for agent ingestion.
- **Scope Control:** It supports --scope=platform or --scope=project. This enforces the "Need to Know" principle. An agent working on ops should not see the project payload, reducing context window saturation and the risk of unauthorized edits to the product core.[1]

7.3 tools/lint/dp.sh (The Contract Enforcer)

- **Function:** Lints the Dispatch Packet.
- **Logic:** It enforces the "Decimal Schema" (3.1, 3.2). It is the rigid validation tool that

caused the friction in DP-OPS-0039 refinement. It ensures that the "Contract" is syntactically correct before a worker is allowed to execute it.[1]

7.4 tools/lint/task.sh (The Logic Guard)

- **Function:** Enforces logic within task files.
- **Hardening:** In DP-OPS-0039, this tool was updated to programmatically enforce the presence of "Closeout pointers" in the execution logic of every task. If a task does not say "Go to Closeout," this linter fails the build.[1]

8. Strategic Recommendations for System Evolution

Based on the forensic analysis of the dump, the thinking logs, and the operational friction observed, the following strategic recommendations are proposed to harden the Stela constitution.

8.1 Resolution of the TASK.md Dirty State

The conflict between PoT.md (Canon) and the dirty state caused by ops/bin/open must be resolved structurally.

- **Recommendation:** Split TASK.md into TASK_TEMPLATE.md (Canonical, Tracked) and ACTIVE_TASK.md (Ephemeral, Ignored).
- **Mechanism:** ops/bin/open should copy the template to the active file and inject the session state there. The AI should be instructed to read ACTIVE_TASK.md for instructions but reference TASK_TEMPLATE.md for the immutable contract. This preserves the "Clean State" of the main branch while allowing the "Dynamic Truth" of the session.

8.2 Enforcement of Fencing via System Prompts

The failure of ChatGPT to fence the DP indicates a weakness in the prompt instructions regarding output format.

- **Recommendation:** Update ops/bin/open (the prompt generator) to include a meta-instruction: *"CRITICAL: Any Dispatch Packet (DP) output must be enclosed in a single Markdown code block (```). Do not output the DP as raw text."*
- **Recommendation:** Update tools/lint/dp.sh to parse for the presence of and markers, rejecting any input that lacks these explicit boundaries.

8.3 Automating Quantitative Reporting

The requirement for agents to "paste tool outputs" into RESULTS is manual and prone to hallucination or omission.

- **Recommendation:** Develop a tools/report binary. This tool would wrap the verification commands (e.g., tools/report verify.sh), capture the stdout/stderr, and automatically

append it to the RESULTS.md file in a standardized format. This removes "Contractor Discretion" from the reporting loop, ensuring the audit trail is purely forensic.

9. Conclusion

The Stela system is a pioneering implementation of "Statutory Governance" for AI software engineering. By treating the repository as a constitution (PoT.md) and the context as a strictly managed file, it mitigates the primary risks of autonomous agents: drift, hallucination, and scope creep.

The friction points identified in this analysis—specifically the "Dirty State" paradox of TASK.md and the "Fencing Failure" of the Contractor—are not failures of the architecture but rather stress tests that reveal the rigor of the system. The worker's complaint about the dirty state proves that the "Anti-Drift" logic is functioning; the system *should* reject unexplained changes. The path forward lies in refining the "Interface Contracts" (like separating the template from the active task) to reduce this friction without compromising the "Absolute Literalism" and "Canonical Truth" that make Stela a viable architecture for high-integrity software development.

Table 1: Stela Filing Doctrine (Physical Laws)

Directory	Type	Purpose	Content Examples
ops/	**Run**	Runtime Kernel, Enforcement	ops/bin/open, ops/lib/manifests/CONTEXT.md
docs/	**Explain**	Legislation, Manuals	PoT.md, docs/MANUAL.md, SoP.md
projects/	**Work**	Payload, Product Code	projects/cms/, projects/api/
storage/	**Trash**	Transient, Non-Canon	storage/handoff/, storage/dumps/

Table 2: Staffing Protocol & Authority

Role	Entity	Key Responsibilities	Operational Constraints
Operator	Human	Intent, Secrets, Commits	Must perform all git push and git merge actions.
Integrator	Lead AI	Governance, Audit, DP Generation	Detects Drift. Rejects PRs. Does NOT write payload code.
Contractor	Guest AI	Logic Execution, Drafting	Absolute Literalism. No Contractions. Scoped edits only.

Table 3: The "Source of Truth" Read-in Order

Order	Document	Purpose
1	PoT.md	**The Constitution.** Physical laws, staffing, jurisdiction. The final authority.
2	SoP.md	**The History.** State of Play ledger. Context on *why* things are the way they are.
3	TASK.md	**The Contract.** Active work surface. Contains the current Dispatch Packet (DP).
4	docs/MAP.md	**The Terrain.** Continuity map for navigating the repository context.

Table 4: System Failure States (Drift Triggers)

Failure Type	Definition	System Response
Canon Drift	Divergence between PoT.md and repo state.	**STOP WORK.** Reject PR.
Context Hazard	Including library/ docs in CONTEXT.md.	**STOP WORK.** Automated lint failure.
Dirty State	Uncommitted changes in main (except TASK.md).	**STOP WORK.** Worker complaint/halt.
Ambiguity	Instructions capable of multiple interpretations.	**STOP WORK.** Seek clarification.

Works cited

1. dump-full-main-a80d295e9.txt